

Aufgabe 1. Betrachten Sie folgende quadratischen Gleichung:



$$x^2 + 2px - q = 0, \quad p, q > 0.$$

Ihre Lösungen lassen sich mithilfe der pq -Formel $x_{1,2} = -p \pm \sqrt{p^2 + q}$ berechnen. Ist man an der größeren der beiden Nullstellen interessiert, erhält man aus der pq -Formel sofort den Algorithmus

$$x_2 = -p + \sqrt{p^2 + q}.$$

Alternativ dazu hat man

$$0 = x^2 + 2px - q = (x - x_1)(x - x_2) = x^2 - (x_1 + x_2)x + x_1x_2$$

und damit

$$x_1x_2 = -q$$

bzw.

$$x_2 = \frac{-q}{x_1}, \quad x_1 = -p - \sqrt{p^2 + q}.$$

Man erhält also den alternativen Algorithmus

- gebe p, q ein
- berechne $x_1 = -p - \sqrt{p^2 + q}$
- berechne $x_2 = \frac{-q}{x_1}$

Implementieren Sie beide Algorithmen in **Python**. Schreiben Sie dazu zwei Funktionen, die p, q als Eingabe- bzw. x als Ausgabeparameter besitzen.

Vergleichen Sie beide Verfahren für $p = 10^2/10^4/10^6/10^7/10^8$ und $q = 1$. Welches Verfahren ist für große p genauer?

Aufgabe 2. Für den natürlichen Logarithmus gilt die folgende Reihenentwicklung:



$$\ln(1+x) = \sum_{k=1}^{\infty} (-1)^{k+1} \frac{x^k}{k}.$$

Implementieren Sie in **Python** eine Funktionen zur Berechnung der abgebrochenen Reihe

$$\ln(1+x) = \sum_{k=1}^n (-1)^{k+1} \frac{x^k}{k}$$

wie folgt:

- $s = 0, x_k = x$
- wiederhole für $k = 1, \dots, n$:
 - $s := s + (-1)^{k+1} \frac{x_k}{k}$
 - $x_k := x_k \cdot x$

Berechnen Sie die Werte für $x = 0.9$ und $n = 50$ indem Sie die **NumPy** Datentypen `float16` und `float64` verwenden. Vergleichen Sie die Ergebnisse mit denen des Logarithmus `log` aus **NumPy**.

Aufgabe 3. Zur Approximation von $\int_a^b f(x) dx$ für stetiges f können als Ersatzprobleme eine der Rechteck-Summen 

$$h \sum_{i=0}^{n-1} f(a + ih), \quad \text{bzw.} \quad h \sum_{i=1}^n f(a + ih)$$

oder die Trapezregel

$$\frac{h}{2} \left(f(a) + 2 \sum_{i=1}^{n-1} f(a + ih) + f(b) \right)$$

benutzt werden, wobei $h = \frac{b-a}{n}$ ist. Die Anzahl n der Unterteilungen des Integrationsbereichs bestimmt dabei den Aufwand des jeweiligen Ersatzproblems und dessen Genauigkeit.

Implementieren Sie die Verfahren als Funktionen in **Python** mit Parametern f, a, b, n . Wenden Sie die Verfahren auf

$$\int_{\frac{1}{10}}^{10} \frac{1}{x^2} dx \quad \text{und} \quad \int_1^2 \ln(x) dx$$

an und vergleichen Sie die Ergebnisse mit der exakten Lösung.

Aufgabe 4. Vektoroperationen werden in **Python** in der Regel sehr viel schneller abgearbeitet als Schleifen über Skalaroperationen. Schreiben Sie die Trapezregel in Vektornotation um, indem Sie 

- einen Vektor mit sämtlichen Stützstellen $a + ih$ erzeugen
- den Integranden f auf diesen Vektor anwenden
- den Vektor der Ergebnisse mit der Funktion **sum** geeignet aufsummieren.

Aufgabe 5. Um nur unter Verwendung der 4 Grundrechenarten die Wurzel einer positiven reellen Zahl näherungsweise zu berechnen gibt es viele verschiedene Ansätze. 

(a) Man kann die Taylorentwicklung

$$f(x) = \sum_{i=0}^{\infty} \frac{f^{(i)}(x_0)}{i!} (x - x_0)^i$$

von $f(x) = \sqrt{x}$ um einen Punkt x_0 betrachten, die Summation nach k Schritten abbrechen, und

$$y_k = \sum_{i=0}^k \frac{f^{(i)}(x_0)}{i!} (x - x_0)^i$$

als Näherung von $y = f(x) = \sqrt{x}$ verwenden.

Per Induktion kann man zeigen, dass sich die y_k , $k = 0, 1, 2, \dots$, durch die *Iterationsvorschrift*

$$\begin{aligned} a_0 &= y_0 = \sqrt{x_0} \\ a_k &= \left(\frac{3}{2k} - 1 \right) \left(\frac{x}{x_0} - 1 \right) a_{k-1} \\ y_k &= y_{k-1} + a_k \end{aligned} \tag{1}$$

berechnen lassen, d.h. wird der Entwicklungspunkt x_0 in der Taylorreihe so gewählt, dass $\sqrt{x_0}$ bekannt ist, dann können die y_k mit den 4 Grundrechenarten bestimmt werden.

Implementieren Sie (1) und berechnen Sie Näherungen von $y = \sqrt{2}$ (d.h. $x = 2$) für $x_0 = 1$ bzw. $x_0 = 4$ (also Startwerte $a_0 = y_0 = 1$ bzw. $a_0 = y_0 = 2$). Bestimmen Sie die Abweichung der y_k vom exakten Wert. Wie viele Schritte benötigt man, bis die Abweichung kleiner 0.005 ist?

- (b) Das *Heron-Verfahren* zur Näherung von $y = \sqrt{x}$ war bereits in Babylonien ca. 1700 vor Christus bekannt, wurde aber nach dem Mathematiker Heron von Alexandria, der im ersten Jahrhundert nach Christus lebte, benannt. Bei diesem Iterationsverfahren gewinnt man aus einer Näherung y_{k-1} eine verbesserte Näherung y_k durch die Iterationsvorschrift

$$y_k = \frac{1}{2} \left(y_{k-1} + \frac{x}{y_{k-1}} \right).$$

Implementieren Sie das Verfahren und berechnen Sie analog zu oben Näherungen zu den Startwerten $y_0 = 1$ bzw. $y_0 = 2$. Bestimmen Sie wieder die Abweichung der y_k vom exakten Wert. Vergleichen Sie die Ergebnisse mit denen aus dem ersten Aufgabenteil.